

KAPITEL 4: CSS

LERNZIELE

- Die grundsätzliche Funktionsweise von CSS erklären können.
- Die Syntax von CSS kennen und anwenden können.
- Unterschiedliche Arten von Selektoren anwenden können.
- Wichtige Möglichkeiten von CSS kennen und anwenden können.
- Das Prinzip von Responsive Webdesign erklären und anwenden können.

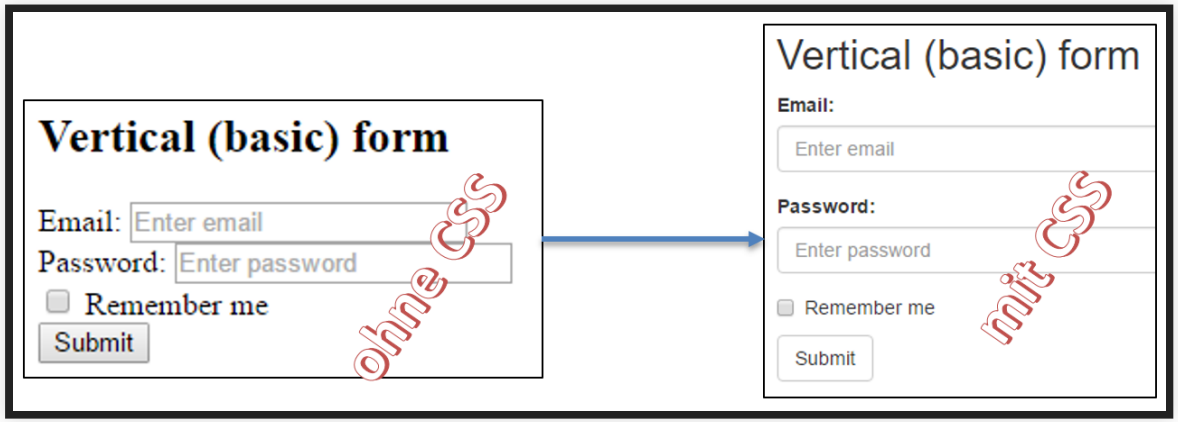
4.1 ÜBERBLICK

In Kapitel 3 wurde erkläutert, dass Inhalt und Style/Layout getrennt spezifiziert werden. Nach der Besprechung von HTML für den Inhalt werden nun die wesentlichen Möglichkeiten von CSS erläutert.

Beachte: Aufgrund der Vielzahl an Features wird hier lediglich ein Ausschnitt besprochen!

Was ist CSS?

CSS ist eine Sprache zur Beeinflussung der Art und Weise, wie Inhalte (=Elemente) eines HTML-Dokuments dargestellt werden:



CSS verfolgt drei wichtige Ziele:

1. exakte Layout-Definition
2. Anpassung an Ausgabemedien (z.B. Browser, Print)
3. zentrales Layout-Management

ENTWICKLUNG VON CSS

Wichtige Meilensteine in der Entwicklung von CSS (siehe auch hier):

- CSS level 1 (1996): Fonts, Colors, Spacing
- CSS level 2 "CSS 2.1" (2006): u.a. absolute Positionierung, automatische Nummerierung, Seitenumbrüche
- CSS level 3 (2011): u.a. abgerundete Ecken, Schatten, Farbverläufe, Animationen, mehrspaltige Darstellung, FlexBox Layoutmanagement

Wie lernt man CSS?

- Nach dem ersten Kennenlernen der wesentlichen Funktionsweise und Sprachelemente hier im Skript vertieft man seine Kenntnisse am besten anhand von Beispielen bzw. während der Umsetzung von Webseiten.
- Neben den hier gezeigten Beispielen findet man Beispiele z.B. auf W3Schools.
- Spezielle Anforderungen können typischerweise nach einer Suche im WWW gelöst werden.

4.2 EINBINDEN VON CSS

CSS wird in Form von Regeln angegeben.

Beispiel: rote Schrift und schwarzer Rand für alle Absätze

```
p {
    color: red;
    border: 1px solid black;
}
```

Die Regel beginnt mit einem sogenannten Selektor, hier wird der Absatz ausgewählt...

... und enthält die Anpassungen für das selektierte Element in den geschweiften Klammern.

Im Folgenden werden die **drei** verschiedenen Arten gezeigt, mit denen CSS-Regeln in einem HTML-Dokument genutzt werden können.

Möglichkeit 1: CSS "inline" verwenden

- Die Style-Spezifikation wird direkt in einem HTML-Element über das style-Attribut angegeben.
- Beachte: Es überschreibt alle anderen Style-Vorgaben!

Beispiel: Der Absatz soll eine dünne Linie als Rand haben

```
<p style='color: red; border: 1px solid black;'>
    Dies ist eine Textzeile
</p>
```

Hier wird erneut Farbe und Rahmen verändert, wie im Beispiel zuvor. Der Selektor ist hier nicht notwendig, da immer das aktuelle Element gemeint ist.

Möglichkeit 2: CSS im Header

Im Header können CSS-Regeln innerhalb eines <style> ... </style>-Blocks definiert werden.

```
<!DOCTYPE html>
<html>
<head>
    <style>

p {

color: red;

border: 1px solid black;
        }

</style>
</head>
<body>
    <p>Dies ist eine Textzeile</p>
</body>
</html>
```

Ein `style`-Tag erlaubt es die Regeln direkt im HTML-Dokument einzubetten.

Möglichkeit 3: Externes Style-Sheet

CSS-Regeln werden in eine oder mehrere Dateien mit Endung ".css" ausgelagert und in die HTML-Dateien eingebunden.

Beispiel: Einbinden des Stylesheets "mystyle.css"

```
<!DOCTYPE html>
<html>
<head>
    <link rel="stylesheet" type="text/css" href="mystyle.css">
</head>
<body>
    <p>Dies ist eine Textzeile</p>
</body>
</html>
```

Inhalt von "mystyle.css" wäre dann:

```
p {
    color: red;
    border: 1 px solid black;
}
```

Für unterschiedliche Ausgabemedien (z.B. Webseite, Druck) gibt es das **media**-Attribut.

Beispiel: Layout-Styles für den Druck der Seite stehen in "druck.css"

```
<link rel="stylesheet" type="text/css" href="mystyle.css">
<link rel="stylesheet" type="text/css" media="print" href="druck.css">
```

Wann nutzt man welche Möglichkeit?

- inline-Style sollte wegen der fehlenden Wiederverwendbarkeit nur zu Testzwecken verwendet werden.
- Style-Blöcke sind besser wiederverwendbar, da sie im gesamten Dokument gelten.
- Externe Stylesheets sind wegen der optimalen Wiederverwendbarkeit und der Möglichkeit zur zentralen Pflege zu bevorzugen!

ÜBUNG: PLATZIERUNG VON CSS-REGELN

Wo sollten Style-Tags in einem HTML-Dokument platziert werden?

- [ ] Im `<body>`-Element
- [ ] Im `<head>`-Element
- [ ] Am Ende des Dokuments, kurz vor dem schließenden `</html>`-Tag
- [ ] Direkt nach dem öffnenden `<html>`-Tag

ÜBUNG: INLINE VS. EXTERN

Erklären Sie den Unterschied zwischen inline Styles und externen Style-Sheets. Nennen Sie je einen Vor- und Nachteil beider Methoden.

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

4.3 GRUNDLAGEN CSS SELEKTOREN

CSS-Regeln spezifiziert man prinzipiell folgendermaßen:

```
selektor1, selektor2, ..., selektorN {
    CSS-property1: value1;
    ...
    CSS-propertyN: valueN;
}
```

- Ein Selektor gibt an, für welche Elemente die folgenden Regeln gelten sollen.
- Neben einfachen Selektoren (z.B. `p` => alle p-Tags) können auch komplexere Selektoren definiert werden, um genau die zu stylenden Elemente zu finden.
- Jede Regel definiert Werte für Style-Eigenschaften (z.B. color oder border).

Einfaches Beispiel zum CSS-Syntax:

```
div {
    background-color: blue;
    font-size: 20px;
    color: white;
}
```

- Die dargestellte Regel gilt für alle div-Elemente in dem Dokument.
- Die Elemente erhalten die Hintergrundfarbe blau.
- Die Schriftgröße der Elemente soll 20 Pixel betragen.
- Die Schriftfarbe soll weiß sein.

CSS Selektoren

Neben den einfachen Element-Selektoren (z.B. "div") gibt es eine Vielzahl weiterer Selektor-Arten. Folgendes Beispiel gibt einen Überblick:

```
#navigation {
    width: 100px;
}
.error {
    color: red;
}
```

- id-Selektor: das Element mit der id "navigation" soll 100 Pixel breit sein.
- class-Selektor: alle Elemente mit der CSS-Klasse "error" sollen rote Schriftfarbe erhalten. Wie man CSS-Klassen definiert und nutzt, wird später erklärt.

Überblick CSS-Selektoren (Fortsetzung)

```
div > p {
    border: 1px solid green;
}
a:visited {
    color: lightgray;
}
```

- Kontext-Selektor: alle p-Elemente, die direkte Kinder eines div-Elements sind, sollen einen grünen Rand haben.
- Pseudoklassen-Selektor: alle besuchten Links sollen hellgrau dargestellt werden.

ID-ATTRIBUT SELEKTOR

```
#someId {
    /*...*/
}
```

Id-Attribut Selektor beginnt mit einem # gefolgt von der entsprechenden Id, die in einem HTML-Element verwendet wird, z.B. `<div id="someId">`.

ÜBUNG: ID-SELEKTOR SYNTAX

Welcher CSS-Selektor sollte verwendet werden, um ein Element mit einer bestimmten ID auszuwählen?

- ☐ `#header` für die ID header
- ☐ `foobar` für die ID foobar
- ☐ `.someId` für die ID someId
- ☐ `[id="ref"]` für die ID ref

CLASS-ATTRIBUT-SELEKTOR

```
.myClass {
    /*...*/
}
```

Class-Attribut Selektor beginnt mit einem `.` gefolgt von der entsprechenden Klasse, die in einem HTML-Element verwendet werden kann, z.B. `<div class="myClass">`.

ÜBUNG: KLASSEN-SELEKTOR SYNTAX

Welcher CSS-Selektor sollte verwendet werden, um ein Element mit einer bestimmten Klasse auszuwählen?

- ☐ `[class="box"]` für die Klasse box
- ☐ `div.main` für die Klasse main eines DIV elements
- ☐ `.content` für die Klasse content
- ☐ `#main` für die Klasse main



So viele Möglichkeiten - welche soll ich nutzen?

- Das World Wide Web-Konsortium empfiehlt die Verwendung von CSS-Klassen.
- Id-Selektoren sollten stets nur ein Element stylen, obwohl Id's mehrfach vergeben werden können. Problematisch bei der Verwendung von gleichen Id's für mehrere Elemente ist, dass die später bei JavaScript häufig genutzte Suche nach der eindeutigen Id eines Elements in solchen Fällen fehlt schlägt.

ÜBUNG: TAG-, KLASSEN- UND ID-SELEKTOREN

Wann würden Sie zu Tag-, Klassen- oder ID-Selektoren greifen?

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

ATTRIBUT-SELEKTOR

Attribut-Selektoren können *noch* spezifischer verwendet werden, wie folgende Beispiele zeigen:

CSS	Wirkung
a[target] {...}	Style für Links mit "target"-Attribut
a[target="_blank"] {...}	Style für Links, die in neuem Fenster öffnen
[title~=flower] {...}	Style für Elemente, deren title-Attribut "flower" enthält

Für das letzte Beispiel gibt es hier eine eine Anwendung.

ÜBUNG: ATTRIBUT-SELEKTOR SYNTAX

Welcher CSS-Selektor wählt ein Texteingabefeld aus?

```
<label for="test">Beispiel: </label>
<input name="beispiel" id="test" value="..." type="text">
```

- [ ] `[input="text"]`
- [ ] `input[type="text"]`
- [ ] `input.text`
- [ ] `#text`

UNITS

CSS hat eine vielzahl möglicher Einheiten, die eine Länge ausdrücken können. Attribute wie `width`, `margin`, `padding` und `font-size` können Längen verwenden.

```
div {
    width: 10px;
}
```

Absolute Längen

- SI-Einheiten und Imperial System: `cm`, `mm`, `in` (1in = 96px = 2.54cm)
- Pixel, Punkte und Picas: `px` (1px = 1/96th of 1in), `pt` (1pt = 1/72 of 1in), `pc` (1pc = 12 pt)

Relative Längen (Auszug)

- `em`: Relativ zur aktuellen Schriftgröße des umschließenden Elements (`2em` meint 2-mal die Größe der aktuellen Schriftgröße)
- `rem`: Relativ zur Schriftgröße des Wurzelelements
- `vw`: Relativ zu 1% der Breite des aktuellen Viewports
- `vh`: Relativ zu 1% der Höhe des aktuellen Viewports
- `%`: Relativ zu einem übergeordneten Element (mit Größenangaben)

Eine vollständige Liste findet man z.B. bei W3Schools.

4.4 CSS EIGENSCHAFTEN

In diesem Abschnitt werden häufig benötigte Möglichkeiten, die CSS bietet, besprochen:

- Schriftfarbe und Hintergrundfarbe
- Border, Margin, Padding: Rahmen und Abstände innerhalb und außerhalb eines Elements
- Höhe und Breite eines Elements setzen
- CSS Box-Modell: Wie kommt die Größe des Rechtecks (=Box) eines Elements zustande?
- CSS Texteigenschaften: Möglichkeiten zum Stylen von Text
- CSS Fonts: Zeichensätze
- display-Eigenschaft: kontrolliert ob und wie ein Element prinzipiell dargestellt wird
- Alignment: Horizontale und vertikale Anordnung von Elementen

SCHRIFT- UND HINTERGRUNDFARBE

```
<h1 style='color: magenta;'>Wo braucht man Farbe?</h1>
<p style='color: #00ff00;'>für den Text</p>
<p style='background-color:#0000ff;
    color: rgb(255, 255, 255);'>
    ...aber auch für den Hintergrund
</p>
```

- mit "color" bzw. "background-color" können Schrift- und Hintergrundfarbe gesetzt werden.
- Farbwerte können vordefinierte Farbnamen (s. hier), Hex- sowie RGB-Werte sein.
- auch Farbverläufe sind möglich, siehe z.B. hier
- RGB- und HEX-Syntax spezifizieren 3 Farbwerte von 0 - 255 für die drei Grundfarben rot, grün und blau

BORDER, MARGIN UND PADDING

```
p {
    border: 1px solid black;
    margin: 10px 5px 10px 5px;
    padding-left: 5px;
    padding-right: 15px;
    padding-top: 20px;
    padding-bottom: 30px;
}
```

- Mit `border` wird die Randbreite, die Linienart und die Linienfarbe definert.
- Mit `margin` wird der Abstand eines Elements zu anderen Elementen festgelegt.
- Mit `padding` wird der Abstand zwischen dem Rand und dem Inhalt eines Elements festgelegt.

Border

Steuert den Rahmen (Farbe, Breite und Stil) eines Elementes. Hierfür können die Eigenschaften getrennt angegeben werden oder kombiniert.

```
#something {
    border-width: 2px;
    border-style: solid;
    border-color: blue;
}
```

```
#something {
    border: 2px solid blue;
}
```

Alle Seiten können darüber hinaus unabhängig voneinander über `border-left`, `border-right`, `border-top` und `border-bottom` konfiguriert werden, sowie erneut die einzelnen Eigenschaften direkt, z.B. `border-left-style`.

Margin

Definiert den Außenabstand eines Elementes (außerhalb von Border).

```
#something {
    margin: 10px;                 /* alle Seiten */
    margin: 10px 20px;            /* vertikal horizontal */
    margin: 10px 20px 30px;       /* oben horizontal unten */
    margin: 10px 20px 30px 40px; /* oben rechts unten links (Uhrzeigersinn)
}
```

Einzelne Seiten können auch direkt gesteuert werden:

```
#something {
    margin-top: 10px;
    margin-right: 20px;
    margin-bottom: 30px;
    margin-left: 40px;
}
```



Besonderheiten:

- Margin kann negative Werte annehmen (z.B. `margin-top: -10px;`)
- Vertikale Margins zwischen Elementen können kollabieren (Margin Collapsing): Der größere Wert wird verwendet
- `margin: 0 auto;` zentriert ein Block-Element horizontal (wenn width gesetzt ist)

Padding

Definiert den Innenabstand eines Elementes (zwischen Border und Inhalt).

```
#something {
    padding: 10px;            /* alle Seiten */
    padding: 10px 20px;       /* vertikal horizontal */
    padding: 10px 20px 30px; /* oben horizontal unten */
    padding: 10px 20px 30px 40px; /* oben rechts unten links */
}
```

Konfiguration der einzelnen Seiten:

```
#something {
    padding-top: 10px;
    padding-right: 20px;
    padding-bottom: 30px;
    padding-left: 40px;
}
```

Besonderheiten:

- Padding kann keine negativen Werte annehmen
- Hintergrundfarbe und Hintergrundbild sind im Padding-Bereich sichtbar
- Padding vergrößert die Gesamtgröße eines Elements (außer bei box-sizing: border-box;)

Box-Model Visualisierung

```
.box {
    background-color: lightblue;
    border: 5px solid darkblue;
    margin: 20px;
    padding: 15px;
    width: calc(100% - 30px - 10px - 40px);
}
.outer {
    background-color: lightgreen;
    width: 100%;
}
```

Die dargestellten `div`-Elemente zeigen einen 5px Border und einen 15px Innenabstand (Padding).



Beobachtung: Obwohl `margin: 20px` definiert ist, zeigt sich der Margin-Effekt unterschiedlich:

- **Links und rechts:** Der Margin ist sichtbar (20px Abstand zum grünen Container)
- **Oben und unten zwischen den Boxen:** Nur 20px statt 40px!

Dies ist das **Margin Collapsing** Phänomen: Vertikale Margins benachbarter Block-Elemente überlappen sich, und es wird nur der größere Wert verwendet. Da beide Boxen `margin: 20px` haben, entsteht zwischen ihnen nur ein Abstand von 20px (nicht 20px + 20px = 40px).

Hinweis: *Margin Collapsing tritt nur bei vertikalen Margins auf, nicht bei horizontalen.*

ÜBUNG: MARGIN VS. PADDING

Welche der folgenden Aussagen in CSS sind korrekt?

- [ ] Margin kann negative Werte annehmen, padding hingegen nicht.
- [ ] Margin wird verwendet, um den Abstand zwischen Textelementen innerhalb eines Containers zu kontrollieren.
- [ ] Margin definiert den Innenabstand eines Elements, während padding den Außenabstand festlegt.
- [ ] Hintergrundfarbe oder Hintergrundbild eines Elements ist im Bereich des Padding sichtbar

HÖHE UND BREITE

```
.foo {
    height: 100px;
    width: 300px;
}
```

- `height` und `width` setzen Höhe und Breite
- Mögliche Werte: bei `auto` setzt der Browser Werte aufgrund des Inhalts; sonst px- oder %-Wert
- `max-height` / `max-width`: maximale Höhe bzw. Breite eines Elements (analog `min-height` etc.)

Tipp: `overflow` bei `div3`: scroll bewirkt Scrollbar, hidden verbirgt nicht passenden Inhalt

DAS CSS BOX-MODELL

```
.foo {
    box-sizing: border-box;
}
```

- Ein Boxmodell legt fest, wie die finale Größe eines Elements bestimmt wird und wird mit `box-sizing` festgelegt.
- Box-Modelle: `content-box` (default) und `border-box`
- Breite content-Box = content-width + Padding (li.+re.) + Border (li.+re.)
- Breite border-Box = width-Angabe

ÜBUNG: BOX-SIZING

Was bewirkt `box-sizing: border-box;`*?*

- [ ] Padding und Border werden in die Gesamtbreite/-höhe eingerechnet
- [ ] Margin wird in die Gesamtbreite/-höhe eingerechnet
- [ ] Es entfernt den Rahmen von der Box
- [ ] Es fügt einen Rahmen zur Box hinzu

CSS TEXTEIGENSCHAFTEN

- `text-align`: Ausrichtung von Text (left, right, center, justify)
- `text-decoration`: Linien über/unter/durch Text (underline, overline, line-through, none)
- `text-indent`: Einrückung der ersten Zeile eines Absatzes
- `line-height`: Zeilenhöhe (als Zahl = Multiplikator der Schriftgröße, oder absolute Angabe)
- `text-transform`: Groß-/Kleinschreibung umwandeln (uppercase, lowercase, capitalize)
- `letter-spacing`: Abstand zwischen Buchstaben
- `word-spacing`: Abstand zwischen Wörtern

Tipp: line-height: 1.5 (ohne Einheit) ist besser als line-height: 24px, da es sich automatisch an die Schriftgröße anpasst!

Beispiele zum Alignment von Text

Beispile zur Text-Decoration



Beispiel zum Indent

Beispiele zur Line-Height

Beispiele zu Text Transform

Beispiele zu Letter- und Word-Spacing

SETZEN DES ZEICHENSATZES

Wichtige Font-Eigenschaften:

- `font-family`: Schriftart (mit Fallback-Optionen)
- `font-size`: Schriftgröße (px, em, rem, %)
- `font-weight`: Schriftstärke (normal, bold, 100-900)
- `font-style`: Schriftstil (normal, italic)

Tipp: 1em entspricht der aktuellen Schriftgröße des Elternelements, typischerweise 16px. Besser: rem (root em) bezieht sich immer auf die Wurzel-Schriftgröße.

Beispiel: Font-Family

Die Font-Family wird mit Fallback-Optionen angegeben: Der erste verfügbare Font wird verwendet.

Beispiel: Font-Size

- Fest konfigurierte 20px
- Schriftgröße in Relation zum Elternelement (div), 16px * 1,5 = 24px
- Schriftgröße in Relation zum Wurzelelement (html), 24px * 1,5 = 36px

Beispiel: Font-Weight

Beispiel: Font-Style

ALIGNMENT VON ELEMENTEN

Wichtige Ausrichtungsmethoden:

- **Horizontal zentrieren** (Block-Elemente): `margin: auto` (erfordert definierte Breite)
- **Text zentrieren**: `text-align: center`
- **Bild zentrieren**: `display: block` + `margin: auto`
- **Float-Positionierung**: `float: left` / `float: right` (Element umfließen lassen)
- **Vertikal zentrieren** (Text): `padding` oder `line-height`

Hinweis: Für modernes Layout sind Flexbox und Grid oft besser geeignet als float

Beispiel: Horizontales Zentrieren von Block-Elementen

Ein Block-Element wird mit `margin: auto` *horizontal zentriert (benötigt eine definierte* `width`*).*

Beispiel: Bild zentrieren

Bilder sind standardmäßig inline-Elemente. Mit `display: block` *werden sie zu Block-Elementen und können mit* `margin: auto` *zentriert werden.*



Beispiel: Float-Positionierung

Mit `float` werden Elemente aus dem normalen Dokumentfluss genommen, andere Inhalte fließen drum herum.

Beispiel: Vertikales Zentrieren von Text

Zwei einfache Methoden für vertikale Zentrierung: `padding` *(oben/unten gleich) oder* `line-height` = `height` *(nur für einzeiligen Text).*

DISPLAY-PROPERTY

Wichtige Display-Werte:

- `display: none` – Element wird komplett ausgeblendet (kein Platz im Layout)
- `display: inline` – Inline-Element wie `<span>` (keine width/height möglich)
- `display: block` – Block-Element wie `<div>` (beginnt auf neuer Zeile)
- `display: inline-block` – Inline-Element, aber width/height möglich
- `display: flex` – Flexbox-Container (modernes 1D-Layout)
- `display: grid` – Grid-Container (modernes 2D-Layout)

Tipp: Für moderne Layouts sind `flex` und `grid` meist besser geeignet als klassische Methoden.

Beispiel: display: none vs. sichtbar

Mit `display: none` wird das Element vollständig ausgeblendet und nimmt keinen Platz mehr ein.

Beispiel: inline vs. block vs. inline-block

Unterschiede:

- `inline`: Bleibt im Textfluss, width/height werden ignoriert
- `block`: Neue Zeile, width/height funktionieren
- `inline-block`: Bleibt im Textfluss, aber width/height funktionieren

Beispiel: Flexbox (display: flex)

Flexbox ist ideal für eindimensionale Layouts (Reihe oder Spalte). Mehr zu Flexbox: CSS-Tricks Guide

Beispiel: Grid (display: grid)

Grid ist perfekt für zweidimensionale Layouts (Zeilen und Spalten). Mehr zu Grid: CSS-Tricks Guide

ÜBUNG: INLINE-BLOCK

Was bewirkt `display: inline-block;`*?*

- [ ] Das Element wird als Tabelle dargestellt
- [ ] Das Element wird immer in einer neuen Zeile angezeigt
- [ ] Das Element verhält sich wie ein Inline-Element, aber akzeptiert Breite und Höhe
- [ ] Das Element wird unsichtbar



4.5 FORTGESCHRITTENE CSS SELEKTOREN

KONTEXT-SELEKTOR

Häufig sollen Elemente gestyle't werden, die innerhalb eines Elements geschachtelt sind. Hierfür gibt es Kontext-bezogene Selektoren, diese werden anhand mehrerer Beispiele erklärt.

Der Vorteil der Verwendung von Kontext-Selektoren ist, dass miteinbezogen wird, in der welchem Element-Kontext sich ein Element befindet. Die Verwendung von Attribut-Selektoren (etwa "id" oder "class") am Element selbst ist dabei nicht notwendig.

Übersicht zu Kontext-Selektoren

1. Der Kind-Selektor:

```
parent-selector child-selector { ... }
```

2. Der "direkte Kinder"-Selektor:

```
parent-selector > child-selector { ... }
```

3. Der "direkter Nachfolger"-Selektor:

```
parent-selector + sibling-selector { ... }
```

4. Der Nachfolger-Selektor:

```
parent-selector ~ sibling-selector { ... }
```

Hinweis: Sowohl Vater- als auch Kontext-Element können aus den bereits vorgestellten Tag-, Id-, Klassen- und Attribut-Selektoren zusammengesetzt werden.

ÜBUNG: KONTEXT-SELEKTOREN

Welche Aussagen für Kontext-Selektoren sind korrekt?

- ☐ `div + p` wählt vorangegangene p-Elemente eines div-Elements aus
- ☐ `div ~ p` wählt alle folgenden p-Elemente auf derselben Ebene aus
- ☐ `div p` wählt alle Vorfahren eines div-Elementes aus, welche einen p-Tag haben
- ☐ `div > p` wählt direkte Kinder eines div-Elements aus, die den p-Tag haben

PSEUDOKLASSEN

Pseudoklassen werden genutzt, um spezielle Zustände von Elementen für die Selektion zu nutzen. Beispielsweise, wenn die Maus über ein Element fährt, oder ein Link angeklickt wurde.

Im Folgenden werden einige häufig genutzte Pseudoklassen am Beispiel erklärt, eine Liste der Pseudoklassen findet man z.B. hier.

CSS	was wird gestyle't ?
a:link { color: blue }	noch nicht besuchte Links
a:visited { color: green }	besuchte Links
div:hover { color: yellow }	Maus fährt über div

Neben Zuständen wie hover und visited gibt es sogenannte Baum-Struktur-Pseudoklassen:

- `:first-child`: Trifft auf Elemente zu, die das erste Kind-Element sind
- `:last-child`: Trifft auf Elemente zu, die das letzte Kind-Element sind
- `:empty`: Trifft auf Elemente zu, die keine Kinder haben
- `:nth-child(An+B)`: Trifft auf Kinder zu die das An+B'te Kind sind, z.B. `:nth-child(2n)` für jedes gerade zweite Kind und `:nth-child(2n+1)` für jedes ungerade zweite Kind

STYLING FÜR LINKS

```
a:link { color: red; }
a:visited { color: green; }
a:hover { color: hotpink; }
```

- Pseudoklassen :link, :visited, :hover, :active zur Spezifikation des Aussehens unbesuchter, besuchter, „gehoverter“ (Maus fährt über Link) bzw. selektierter Links
- Links können auch als Buttons dargestellt werden, entweder inline (1. Button) oder als Block (2. Button)
- Die Werte der display-Eigenschaft werden später erklärt

STYLING FÜR TABELLEN

- Gestaltung der Tabellen-Umrandungen mit der border-Eigenschaft, `border-collapse` vermeidet doppelte Linien
- Festlegen der Breite und Höhe der Tabelle bzw. der Tabellenzeilen und -spalten (heigth, width)
- Alignment: Festlegung der horizontalen und vertikalen Anordnung der Zelleninhalte (`text-align`)

Tabellen können folgendermaßen gestyle't werden (Fortsetzung):

- Padding: Festlegen des Abstands zwischen den Rändern und dem Text
- Table-Hover: Hervorhebung von Zeilen beim Überfahren mit der Maus (tr:hover)
- Alternierende Hintergrundfarbe für Zeilen mit `nth-child(odd)` / `nth-child(even)`
- Responsive: Automatische Scrollbar, falls die Breite bzw. die Höhe des Browserfensters nicht ausreicht (`overflow-x`)

PSEUDOELEMENTE

Neben Pseudoklassen, welche Zustände zu adressieren, gibt es Pseudoelemente welche den Zugriff auf spezielle Teile eines Elements erlaubt. Dies ist z.B.:

- `first-line`: Erste Zeile eines Textes
- `first-letter`: Erstes Zeichen eines Textes
- `selection`: Auswahl eines Textes
- `after`: Vor einem Text
- `before`: Nach einem Text

KOMPLEXE "REAL-LIFE"-BEISPIELE AUS BOOTSTRAP

Bootstrap ist das weltweit meistgenutzte CSS-Framework. Urprünglich wurde es für Twitter entwickelt, jetzt nutzen viele Webanwendungen die vordefinierte Style-Möglichkeiten und Funktionalitäten, die Bootstrap "out-of-the-box" bietet.

Im Folgenden werden einige Beispiele aus dem Stylesheet von Bootstrap besprochen.

Bootstrap Beispiel 1

```
.table-hover > tbody > tr.warning:hover > th {
    background-color: #faf2cc;
}
```

Erläuterung: setze die Hintergrundfarbe auf den Hex-Wert `#faf2cc` für alle `th`-Elemente, die direkte Kinder von "gehoverten" `tr`-Elementen mit der Klasse `warning` sind, die wiederum direkte Kinder von `tbody`-Elementen sind, die direkte Kinder von Elementen mit der Klasse `table-hover` sind.

Bootstrap Beispiel 2

```
input[type="radio"],
input[type="checkbox"] {
    margin: 4px 0 0;
    line-height: normal;
}
```

Erläuterung: setze Margins und Zeilenhöhe für Radio- und Checkboxen.

Bootstrap Beispiel 3

```
.open > .dropdown-toggle.btn-success {
  color: #fff;
  background-color: #449d44;
  border-color: #398439;
}
```

Erläuterung: Style für direkte Kinder mit Klasse `dropdown-toggle` **und** `btn-success` setzen eines Eltern-Elements mit der Klasse `open`.

ÜBUNG: KOMPLEXER CSS-SELEKTOR

Gegeben ist folgender Selektor, erklären Sie, was genau dieser macht:

```
nav.main-navigation > ul li a[href^="https://"]:hover::after
```

Geben Sie eine eigene Antwort, eine Beispielantwort wird nach dem Absenden angezeigt

REGELPRIORISIERUNG

Gelten mehrere CSS-Regeln für ein HTML-Element, nutzt der Browser folgendes Verfahren. Es berechnet für jede Regel r einen Wert w_r:

Start mit $w_r := 0$

w_r+=	**Bedingung: r befindet sich in ...**
1000	einer inline style-Anweisung
100	einem id-Selektor
10	einer CSS-Klasse, einem Attribut (z.B. input[type=button]) oder einer Pseudo-Klasse (z.B. a:link)
1	einem Element-Selektor (z.B. h1 { … })

Angewandt wird die Regel mit dem größten w-Wert. Haben alle anwendbaren Regeln den gleichen w-Wert, so gewinnt die zuletzt definierte.

ÜBUNG: REGELPRIORISIERUNG (1)

Gegeben sind folgender HTML- und CSS-Snippet.

```html
<header id="header">
  <nav class="nav">
    <ul>
      <li><a href="https://example.com">Link</a></li>
    </ul>
  </nav>
</header>
```

```css
#header .nav li a { color: blue; }
body header nav ul li a { color: green; }
li a[href]:hover { color: red; }
```

Hinweis: Probieren Sie dies in einem entsprechenden Experiment aus.

Welche Farbe hat der Link im normalen Zustand?

- [ ] Grün
- [ ] Rot
- [ ] Blau
- [ ] Schwarz (Standard)

ÜBUNG: REGELPRIORISIERUNG (2)

Gegeben sind die selben HTML- und CSS-Snippets. Welche Farbe hat der Link, wenn der Mauszeiger darüber schwebt (hover)?

- [ ] Blau
- [ ] Rot
- [ ] Schwarz (Standard)
- [ ] Grün

ÜBUNG: REGELPRIORISIERUNG (3)

Welche der CSS-Regeln hat die höchste Priorität?

- [ ] `body header nav ul li a`
- [ ] Alle haben die gleiche Spezifizität
- [ ] `li a[href]:hover`
- [ ] `#header .nav li a`



4.6 RESPONSIVE WEB DESIGN



Webseiten werden heutzutage auf verschiedenen Geräten (Desktop, Tablet, Smartphone) genutzt.

Sinkt die verfügbare Bildschirmbreite, so wird die gesamte Webseite verkleinert (Skalierung)

Dies hat zur Folge, dass die Seite relativ schnell nicht mehr lesbar bzw. bedienbar ist.

Responsive Webdesign (RWD) ist ein Ansatz zur Lösung dieser Problematik:

- Ziel ist eine gute Benutzererfahrung (user experience) bzgl. einer Seite auf allen Endgeräten.
- Die Grundidee ist dabei, die Inhalte und das Seitenlayout an die verfügbare Bildschirmbreite anzupassen.

RWD Ansatz

- Eine Einteilung jeder Seite in Spalten (grid view) erzeugt eine flexible Grundstruktur, bei der jede Spaltengrenze eine mögliche Umbruchstelle darstellt.
- Wird die verfügbare Ausgabegerät-Breite kleiner, bricht man den Teil, der nicht mehr dargestellt werden kann, um.
- Der bei großem Display als ganzes dargestellte Inhalt wird auf zwei oder mehr Zeilen verteilt.

MEDIA QUERY IN CSS

Grundlage zur Realisierung dieses Ansatzes ist der Media Query in CSS. Dieser erlaubt es Regeln zu definieren die z.B. nur ab bis bis zu einer bestimmten Bildschirmbreite angewendet werden. Der Syntax hierfür ist:

```
@media [Medientyp] [and (bedingung) ] {
  ...;
}
```

Ein Beispiel für Regeln, die nur gelten, wenn der Bildschirm kleiner als 500px ist wäre dann zum Beispiel:

```
@media screen and (max-width: 500px) {
  body {
    background-color: red;
  }
}
```

Ein Beispiel für Regeln, die nur gelten, wenn der Bildschirm größer gleich 500px ist wäre dann zum Beispiel:

```
@media screen and (min-width: 500px) {
  body {
    background-color: yellow;
  }
}
```

Gilt die Bedingung des Media Queries, so werden die Regeln mit berücksichtigt. Ist die Bedingung nicht erfüllt, werden die Regeln nicht angewendet.

Wichtig: Sobald die Bedingung erfüllt ist, wirken sich auch wieder die Konzepte der Regelpriorisierung aus. Folgendes Beispiel würde funktionieren (bei <= 500px Bildschirmbreite), da die Regeln gleich priorisiert werden:

```
body {
  background-color: red;
}
@media screen and (max-width: 500px) {
  body {
    background-color: yellow;
  }
}
```

Folgendes Beispiel würde dagegen keine Änderung anzeigen (bei <= 500px Bildschirmbreite), da die erste Regeln höher priorisiert wird:

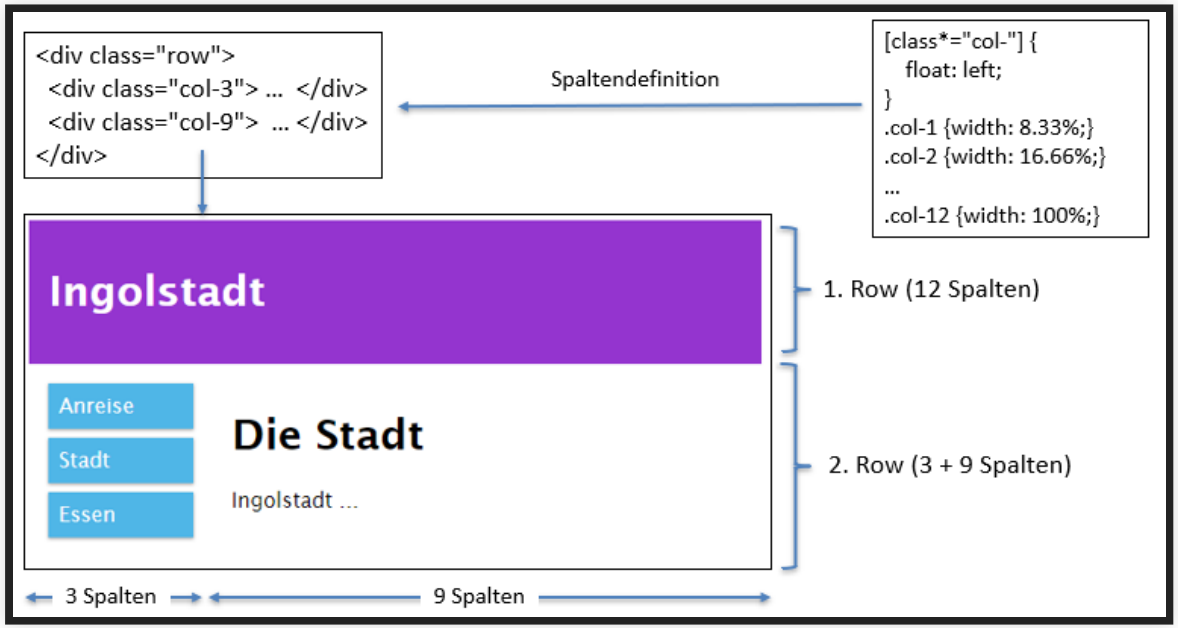
```
html > body {
  background-color: red;
}
@media screen and (max-width: 500px) {
  body {
    background-color: yellow;
  }
}
```

WIE FUNKTIONIERT DAS GRID-SYSTEM?

Ansatz:

- Einteilung in 12 Spalten
- Einteilung in der Höhe durch Zeilen (=div's mit class="row")
- Eine "row" enthält mehrere div's mit einer bestimmten Anzahl an Spalten (3 Spalten z.B. durch class="col-3")
- Wichtig: die Summe der col-*-Angaben muss 12 ergeben!

Im Beispiel:



- Zunächst werden 12 Klassen für die Spalten definiert: col-1, col-2 usf.
- Damit werden 2 row's definiert
  - Die row für den Titel mit 12 Spalten (col-12)
  - Darunter die row für Menü (3 Spalten: col-3) und den Inhalt (9 Spalten: col-9)



Das Layout sieht auf Smartphones nicht gut aus. Wie lösen wir das Problem?

Lösung:

- Ab einer bestimmten Display-Breite (z.B. `768px`) soll ein anderes Layout dargestellt werden.
- Wir definieren einen "breakpoint" bei `768px` mit Hilfe eines Media Queries

```
@media only screen and (max-width: 768px) {
  [class*="col-"] {
    /* alle Klassen mit Präfix “col-” */
    width: 100%;
  }
}
```

- Ab einer Breite von `768px` hat jede Spalte eine Breite von 100% (`col-*: 100%`), d.h. es wird zwischen den Spalten umgebrochen.

RWD-FRAMEWORKS

- RWD-Frameworks bieten das "grid view"-Konzept und Basiskomponenten (Menüs, Formulare, Tabellen, …)
- Vorteile: Aufwand-Ersparnis (Erstellung, Wartung), Browserkompatibilität
- Beispiele:
  - W3C.CSS (http://www.w3schools.com/w3css/default.asp)
  - Bootstrap (http://getbootstrap.com)
  - Google Material (https://material.google.com/)
  - populär: Tailwind (https://tailwindcss.com/)

Die Viewport-Einstellung verhindert, dass eine Seite einfach klein-skaliert wird, wenn man sie auf einem mobilen Gerät betrachtet.

Folgende Einstellung ist in vielen Fällen sinnvoll:

```
<meta
  name="viewport"
  content="width=device-width, initial-scale=1.0, user-scalable=no"
/>
```

Die Details zur Viewport-Einstellung für mobile Webseiten werden z.B. hier erklärt.

ÜBUNG: MEDIA-QUERY

Gegeben ist folgender CSS Media Query.

```css
.menu-items,
.menu-items li {
  display: inline-block;
}
@media screen and (max-width: 600px) {
  .menu-items {
    display: block;
  }
  .menu-items li {
    display: block;
    margin-bottom: 10px;
  }
}
```

Welche Aussagen treffen auf diese Media Query zu?

- [ ] Sie wird angewendet, wenn die Bildschirmbreite größer als 600px ist.
- [ ] Sie ändert die Darstellung der Menüpunkte auf eine vertikale Anordnung für Bildschirme unter 600px Breite.
- [ ] Sie wird angewendet, wenn die Bildschirmbreite 600px oder weniger beträgt.
- [ ] Sie hat keinen Einfluss auf die Darstellung der Menüpunkte.
- [ ] Sie wird angewendet, wenn die Bildschirmbreite genau 600px beträgt.

ÜBUNG: MEDIA QUERY UND REGELPRIORISIERUNG

Gegeben ist folgender CSS-Snippet. Welche Aussagen treffen für ein `<li>`-Element mit der Klasse nav-item innerhalb von `<nav id="main-nav">` bei einer Bildschirmbreite von 500px zu?

```css
.nav-item {
  display: inline-block;
  margin-right: 20px;
}
#main-nav .nav-item {
  font-size: 16px;
}
@media screen and (max-width: 600px) {
  nav .nav-item {
    display: block;
    margin-bottom: 10px;
    font-size: 14px;
  }
}
```

Welche Aussagen treffen bei einer Bildschirmbreite von 500px zu?

- [ ] Seine Schriftgröße beträgt 14px.
- [ ] Es hat einen unteren Außenabstand (margin-bottom) von 10px.
- [ ] Seine Schriftgröße beträgt 16px.
- [ ] Es hat einen rechten Außenabstand (margin-right) von 20px.
- [ ] Es wird als Inline-Block-Element dargestellt.
- [ ] Es wird als Block-Element dargestellt.

4.7 BEST PRACTICE

Was sind bewährte Vorgehensweisen im Zusammenhang mit der Nutzung von CSS?

Es gibt CSS-Spracherweiterungen, z.B. SASS Syntactically Awesome StyleSheets

- SASS erweitert CSS beispielsweise um Variablen und geschachtelte Regeln.
- SASS-Regeln werden durch einen Präprozessor auf CSS-Regeln abgebildet, bevor eine Seite im Browser dargestellt wird.

Einsteiger-Tipps

30 Tipps für Einsteiger findet man hier: http://code.tutsplus.com/tutorials/30-css-best-practices-for-beginners--net-6741.

Viele Tipps rund um die Ausgestaltung von Webseiten findet man z.B. auf http://www.w3schools.com/howto/default.asp

UMSETZUNG VON FORMULAREN

Formulare werden sehr häufig benötigt, daher hier einige Hinweise zur Umsetzung von Formularen:

- Bootstrap unterstützt bei der Formular-Definition, siehe https://www.w3schools.com/bootstrap4/bootstrap_forms.asp
- Lösungen für Formulare ohne ein externes Framework findet man z.B. hier

FLEXBOX

Von JavaFX sind verschiedene Pane-Klassen bekannt, die ihre Inhalte (Panes oder Controls) auf unterschiedliche Art und Weise anordnen mit dem Ziel, das Erstellen von komplexen Layouts zu erleichtern.

CSS Flexbox (Flexible Box Layout) ist ein Layout-Modell in CSS, das ein Layout-Management analog zu HBox, VBox und FlowPane bietet. Mit Flexbox können Entwickler:innen Elemente auf einer Seite in einer Weise anordnen, die sich automatisch an verschiedene Bildschirmgrößen und -orientierungen anpasst. Dies macht Flexbox besonders nützlich für das Erstellen von responsiven Webdesigns.

Ein umfassendes Tutorial zu Flexbox findet man z.B. hier

- Je nach verfügbarer Bildschirm-Breite werden die drei Boxen in einer Reihe dargestellt oder umgebrochen.
- durch Veränderung der Attribute (siehe Kommentare) kann das Verhalten der Flexbox angepasst werden.
- Bitte beachten: die "justify-content"-Attribute sind nur zur Demonstration der Möglichkeiten aufgeführt, d.h. es wird nur ein Wert benötigt!

CSS VARIABLEN

Das Ändern der Grundfarben einer Website ist typischerweise mit Aufwand verbunden, da die Grundfarben für Hintergrund, Schriftfarbe etc. an vielen Stellen auftauchen können.

Mit CSS-Variablen lässt sich diese Situation "zentral" lösen, wie folgendes Beispiel zeigt.

- die Pseudoklasse :root style't das Wurzel-Element des HTML-Baumes. Hier werden die drei Farben über Variablen definiert.
- mit var(--background) wird der Wert der Variable "--background" an beliebiger Stelle genutzt, analog für die beiden übrigen Variablen.
- In der JavaScript-Funktion setMode() werden die drei Variablen je nach neuem Modus gesetzt.

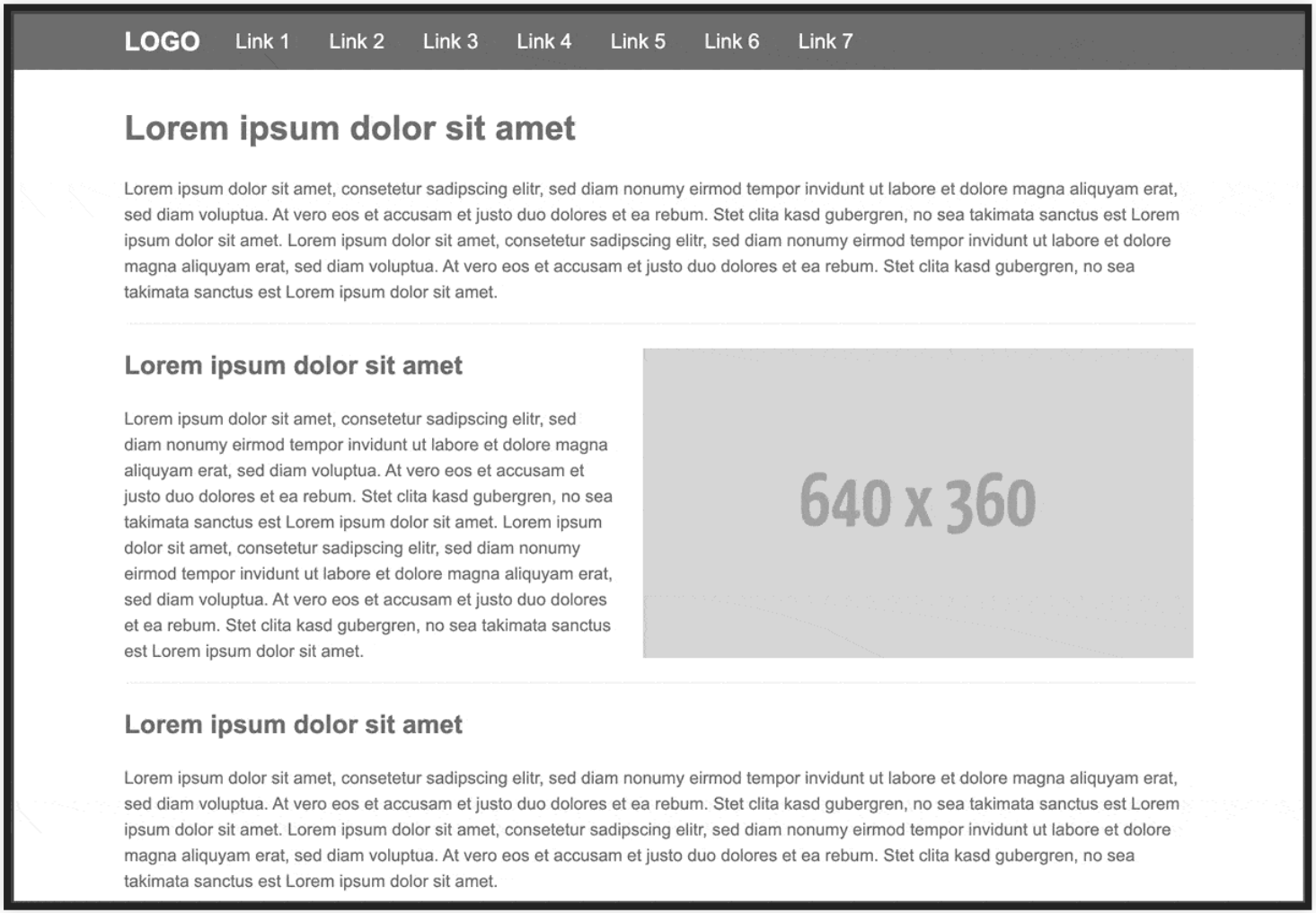
4.8 ÜBUNGEN

ÜBUNG: RESPONSIVE WEBSEITE

Vorgegeben ist ein Layout mit einer Navigation und einem Inhaltstext. Ziel ist, dass sich das Layout sich an die Breite der verfügbaren Oberfläche anpasst.

Nutzen Sie das bereitgestellte Projekt-Archiv css-nav.zip, entpacken Sie dieses und öffnen Sie den Ordner in Visual Studio Code.

Ziel der Anpassungen:



Zur Vorbereitung:

- Öffnen Sie die Seite in einem Browser und verwenden Sie die Entwicklungswerkzeuge um verschiedene Browsergrößen auszuprobieren
- Analysieren Sie die Struktur des Dokumentes, und wie sich die Standardansicht und deren Breiten ergeben

Folgende Anpassungen sind durchzuführen:

- Ergänzen Sie den Meta-Tag für den Viewport, wie er in den Folien vorgestellt wurde
- Sorgen Sie dafür, dass sich der Container in der Navigation, der Inhalt-Container, sowie die Liste und die entsprechenden Einträge anpassen, sobald eine Breite von weniger als `1024px` verfügbar ist.
  - Der Display-Typ und die Breite ist zu ändern
  - Achten Sie auf die Regelpriorisierung
- Sorgen Sie dafür, dass die Position des Bildes nicht mehr neben dem Text positioniert ist, sobald eine Breite von weniger als `756px` verfügbar ist.

